THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

**LAB ASSIGNMENT-5**

Submitted by:
Pranav Mishra
1024150203

2O23

1. Write a simple base class, then a derived class and use objects of both of them in the main function. It will be a simple illustration of inheritance.

```
#include <iostream>
using namespace std;
class Base {
public:
    void show() {
    cout << "This is Base class" << endl;
    }
};
class Derived : public Base {
public:
    void display() {
    cout << "This is Derived class" << endl;
    }
};
int main() {
    Base b;
    Derived d;
    b.show();
    d.show();
    d.display();
    return 0;
}
```

**OUTPUT:**

```
C:\Users\pg664\OneDrive\Do

This is Base class
This is Base class
This is Derived class

--------------------------------
Process exited after 1.437 seconds with return value 0
Press any key to continue . . .
```

2. Practice protected access specifier in inheritance. In the base class declare a variable which is protected and access it in the derived class.

```cpp
#include <iostream>
using namespace std;
class Base {
protected:
    int x = 10;
};
class Derived : public Base {
public:
    void show() {
        cout << "Protected value: " << x << endl;
    }
};
int main() {
    Derived d;
    d.show();
    return 0;
}
```

**OUTPUT:**

```
C:\Users\pg664\OneDrive\Do

Protected value: 10

--------------------------------
Process exited after 1.557 seconds with return value 0
Press any key to continue . . .
```

3. Most of the time we use public mode of inheritance, for example class Derived: public Base{}; Try protected and private access modifiers to understand the difference of various modes of inheritance.

```cpp
#include <iostream>
using namespace std;
class Base {
public:
    int a = 1;
protected:
    int b = 2;
private:
    int c = 3;
};
class PublicDerived : public Base {
public:
    void show() {
        cout << a << " " << b << endl;
    }
};
class ProtectedDerived : protected Base {
public:
    void show() {
        cout << a << " " << b << endl;
    }
};
class PrivateDerived : private Base {
public:
```

```cpp
    void show() {
        cout << a << " " << b << endl;
    }
};
int main() {
    PublicDerived p;
    p.show();
    ProtectedDerived pr;
    pr.show();
    PrivateDerived pv;
    pv.show();
    return 0;
}
```

**OUTPUT:**

```
C:\Users\pg664\OneDrive\Do

1 2
1 2
1 2

--------------------------------
Process exited after 1.908 seconds with return value 0
Press any key to continue . . .
```

4. Various types of inheritances are as shown below. Write small C++ codes for each inheritance type.
   - Single Inheritance

```cpp
#include <iostream>
using namespace std;
class A {
public:
    void showA() {
        cout << "Class A function\n";
    }
};
class B : public A {
public:
    void showB() {
        cout << "Class B function\n";
    }
};
int main() {
    B obj;
    obj.showA();
    obj.showB();
    return 0;
}
```

**OUTPUT:**

```
C:\Users\pg664\OneDrive\Do

Class A function
Class B function

--------------------------------
Process exited after 1.211 seconds with return value 0
Press any key to continue . . .
```

- Multiple Inheritance

```cpp
#include <iostream>
using namespace std;
class A {
public:
    void showA() {
        cout << "Class A\n";
    }
};
class B {
public:
    void showB() {
        cout << "Class B\n";
    }
};
class C : public A, public B {
public:
    void showC() {
        cout << "Class C\n";
    }
};
int main() {
    C obj;
    obj.showA();
    obj.showB();
    obj.showC();
    return 0;
}
```

**OUTPUT:**

```
C:\Users\pg664\OneDrive\Do

Class A
Class B
Class C

--------------------------------
Process exited after 1.52 seconds with return value 0
Press any key to continue . . .
```

- Hierarchical Inheritance

```cpp
#include <iostream>
using namespace std;
class A {
public:
    void showA() {
        cout << "Class A\n";
    }
};
class B : public A {
public:
```

**OUTPUT:**

```
C:\Users\pg664\OneDrive\Do

Class A
Class B
Class C

--------------------------------
Process exited after 1.451 seconds with return value 0
Press any key to continue . . .
```

- Multilevel Inheritance

```cpp
#include <iostream>
using namespace std;
class A {
public:
    void showA() {
        cout << "Class A\n";
    }
};
class B : public A {
public:
    void showB() {
        cout << "Class B\n";
    }
};
class C : public A {
public:
    void showC() {
        cout << "Class C\n";
    }
};
int main() {
    B obj1;
    C obj2;
    obj1.showA();
    obj1.showB();
    obj2.showA();
    obj2.showC();
    return 0;
}
```

**OUTPUT:**

```
C:\Users\pg664\OneDrive\Do

Class A
Class B
Class A
Class C

--------------------------------
Process exited after 1.101 seconds with return value 0
Press any key to continue . . .
```

- Hybrid Inheritance

```cpp
#include <iostream>
using namespace std;
class A {
public:
    void showA() {
        cout << "Class A\n";
    }
};
```

```cpp
class B : public A {
public:
    void showB() {
        cout << "Class B\n";
    }
};
class C : public A {
public:
    void showC() {
        cout << "Class C\n";
    }
};
class D : public B, public C {
public:
    void showD() {
        cout << "Class D\n";
    }
};
int main() {
    D obj;
    obj.B::showA(); // resolve ambiguity
    obj.showB();
    obj.showC();
    obj.showD();
    return 0;
}
```

**OUTPUT:**

```
C:\Users\pg664\OneDrive\Do

Class A
Class B
Class C
Class D

--------------------------------
Process exited after 1.553 seconds with return value 0
Press any key to continue . . .
```

5. How can you use constructors and destructors in C++ inheritance? Write a program to illustrate

```cpp
#include <iostream>
using namespace std;
class Base {
public:
    Base() {
        cout << "Base Constructor\n";
    }
    ~Base() {
        cout << "Base Destructor\n";
    }
};
class Derived : public Base {
public:
    Derived() {
        cout << "Derived Constructor\n";
    }
    ~Derived() {
        cout << "Derived Destructor\n";
    }
};
int main() {
    Derived d;
    return 0;
}
```

**OUTPUT:**

```
C:\Users\pg664\OneDrive\Do

Base Constructor
Derived Constructor
Derived Destructor
Base Destructor

--------------------------------
Process exited after 1.633 seconds with return value 0
Press any key to continue . . .
```

6. Implement a base class Book with attributes title, author, and price. Then, create a derived class Textbook that inherits from Book and adds a new attribute subject. Demonstrate how single inheritance is used to manage the data for general books and textbooks.

```cpp
#include <iostream>
using namespace std;
class Book {
public:
    string title, author;
    float price;
    void getData() {
        cout << "Enter title: ";
        getline(cin, title);
        cout << "Enter author: ";
        getline(cin, author);
        cout << "Enter price: ";
        cin >> price;
    }
    void showData() {
        cout << title << " " << author << " " << price << endl;
    }
};
class Textbook : public Book {
public:
    string subject;
    void getSubject() {
        cin.ignore();
        cout << "Enter subject: ";
        getline(cin, subject);
    }
    void showAll() {
        showData();
        cout << subject << endl;
    }
};
int main() {
    Textbook t;
    t.getData();
    t.getSubject();
    t.showAll();
}
```

**OUTPUT:**

```
C:\Users\pg664\OneDrive\De

Enter title: how to live life
Enter author: prag
Enter price: 120
Enter subject: life
how to live life prag 120
life

--------------------------------
Process exited after 21.78 seconds with return value 0
Press any key to continue . . .
```

7. A software company is creating a program to simulate a car's dashboard. The dashboard needs to display speed, fuel level, and temperature. The speed is controlled by a Speedometer class, the fuel level by a FuelGauge class, and the temperature by a Thermometer class. Implement the three classes: Speedometer, FuelGauge, and Thermometer, each with relevant attributes and methods. Then, create a CarDashboard class that inherits from all three classes to display the combined information on the dashboard. Demonstrate how multiple inheritance is used to build this class.

```cpp
#include <iostream>
using namespace std;
class Speedometer {
public:
    int speed = 60;
};
class FuelGauge {
public:
    int fuel = 50;
};
class Thermometer {
public:
    int temp = 30;
};
class CarDashboard : public Speedometer, public FuelGauge, public Thermometer {
public:
    void display() {
        cout << "Speed: " << speed << endl;
        cout << "Fuel: " << fuel << endl;
        cout << "Temp: " << temp << endl;
    }
};
int main() {
    CarDashboard c;
    c.display();
}
```

**OUTPUT:**

```
C:\Users\pg664\OneDrive\De

Speed: 60
Fuel: 50
Temp: 30

--------------------------------
Process exited after 1.36 seconds with return value 0
Press any key to continue . . .
```

8. You are tasked with creating a system for a library that tracks different types of users. The system needs to handle general user information such as name, ID, and contact details. There are two specific types of users: Student and Teacher. Each type of user has additional attributes, such as grade level for students and department for teachers. Implement a base class LibraryUser with general attributes. Then, create two derived classes Student and Teacher that inherit from LibraryUser and add their own specific attributes. Demonstrate how hierarchical inheritance is applied in this scenario.

```cpp
#include <iostream>
using namespace std;
class LibraryUser {
public:
    string name;
};
class Student : public LibraryUser {
public:
    int grade;
};
class Teacher : public LibraryUser {
public:
    string department;
};
int main() {
    Student s;
    s.name = "Prag";
    s.grade = 10;
    Teacher t;
    t.name = "Sir";
    t.department = "Math";
    cout << s.name << " " << s.grade << endl;
    cout << t.name << " " << t.department << endl;
}
```

**OUTPUT:**

```
C:\Users\pg664\OneDrive\De

Prag 10
Sir Math

--------------------------------
Process exited after 1.369 seconds with return value 0
Press any key to continue . . .
```

9. A logistics company needs a software system to manage its vehicle fleet. All vehicles share common attributes like make, model, and year. Trucks have additional attributes like load_capacity. Furthermore, refrigerated trucks have a special attribute called temperature_control. Implement a base class Vehicle with common attributes. Then, create a derived class Truck that adds the load_capacity attribute. Finally, create another derived class RefrigeratedTruck that inherits from Truck and adds the temperature_control attribute. Demonstrate how multilevel inheritance works in this case.

```cpp
#include <iostream>
using namespace std;
class Vehicle {
public:
    string make;
};
class Truck : public Vehicle {
public:
    int load_capacity;
};
class RefrigeratedTruck : public Truck {
public:
    int temperature;
};
int main() {
    RefrigeratedTruck r;
    r.make = "Tata";
    r.load_capacity = 1000;
    r.temperature = -5;
    cout << r.make << " " << r.load_capacity << " " << r.temperature;
}
```

**OUTPUT:**

```
C:\Users\pg664\OneDrive\De:
Tata 1000 -5
--------------------------------
Process exited after 1.54 seconds with return value 0
Press any key to continue . . .
```

10.You are developing a software system for an academic institution. The institution has various roles like Person, Staff, and Student. A Person has general attributes like name and address. Staff members, who are a type of Person, have additional attributes like employee_id and department. Student, another type of Person, has attributes like student_id and grade. Some Staff members are also Students (e.g., teaching assistants) and need to inherit from both classes. Implement a base class Person with general attributes. Then, create derived classes Staff and Student that inherit from Person and add their specific attributes. Finally, create a TeachingAssistant class that inherits from both Staff and Student. Demonstrate how hybrid inheritance is applied and managed in this scenario.

```cpp
#include <iostream>
using namespace std;
class Person {
public:
    string name;
};
class Staff : public Person {
public:
    int emp_id;
};
class Student : public Person {
public:
    int student_id;
};
class TeachingAssistant : public Staff, public Student {
public:
    void show() {
        cout << Staff::name << endl;
        cout << emp_id << " " << student_id << endl;
    }
};
int main() {
    TeachingAssistant t;
    t.Staff::name = "Prag";
    t.emp_id = 101;
    t.student_id = 202;

    t.show();
}
```

**OUTPUT:**

```
C:\Users\pg664\OneDrive\De:
Prag
101 202

--------------------------------
Process exited after 1.162 seconds with return value 0
Press any key to continue . . .
```